ENSF-381: Full Stack Web Development Laboratory

Ahmad Abdellatif
Department of Electrical & Software Engineering
University of Calgary

Lab 7

Objectives

Welcome to the ENSF381 course lab! In this lab, we will create a Python tool that analyzes the structure and content of web pages. Along the way, we will revisit some key Python basics while working with BeautifulSoup, a powerful library for web scraping. You will get to extract text, find common words, and even visualize the data you gather.

Groups

Lab instructions must be followed in groups **of two students**.

Submission

- You must submit the complete source code, ensuring it can be executed without any modifications.
- If requested by the instructor, you may need to submit additional documentation (e.g., word documents and images).
- The link to the GitHub repository created for this lab must be provided alongside your submission on D2L. If you submit a word document as a part of this lab, include the link to the GitHub repository within that document as well.
- Only one member of the group needs to submit the assignment, but the submission must include the names and UCIDS of all group members.

Deadline

Lab exercises must be submitted by 11:55 PM on the same day as the lab session. Submissions made within 24 hours after the deadline will receive a maximum of 50% of the mark. Submissions made beyond 24 hours will not be evaluated and will receive a grade of zero.

Introduction to BeautifulSoup

BeautifulSoup is a Python library used for parsing HTML and XML documents. It creates a parse tree for the parsed web page, making it easy to extract data from HTML tags like <h1>, <a>, and <p>. It's widely used in web scraping projects.

BeautifulSoup is a powerful and versatile library used for parsing HTML due to its many advantages. One of its key benefits is **its simple syntax**, which makes it easy to extract and manipulate data from HTML documents. Furthermore, BeautifulSoup **offers powerful search features** that allow users to efficiently find specific tags and extract relevant data. BeautifulSoup's flexibility allows it to easily handle imperfect or poorly formatted HTML, making it an ideal tool for web scraping tasks where the source code may not always be well-structured.

Installing BeautifulSoup

To install BeautifulSoup and requests, follow these steps:

1. Open your terminal (Command Prompt, PowerShell, or any command-line tool).
2. Run the following command to install both libraries:

```
pip install beautifulsoup4 requests
```

3. If you encounter issues with installation, consider upgrading pip with:

```
pip install --upgrade pip
```

4. To verify the installation, open a Python shell by typing python and run:

```
import requests
from bs4 import BeautifulSoup
print("BeautifulSoup and requests are installed correctly!")
```

If no errors appear, the libraries are successfully installed and ready to use.

1. Creating a Python Project Using VS Code

- Open Visual Studio Code (VS Code).
- Create a new folder for your project (e.g., WebAnalyzer).
- Open this folder in VS Code by selecting **File → Open Folder**.
- Inside the folder, create a new file called web_analyzer.py.

2. Crawl the UofC Wikipedia webpage

In this step, write code that uses requests to retrieve the webpage content and BeautifulSoup to parse it.

```
import requests
from bs4 import BeautifulSoup

url = "https://en.wikipedia.org/wiki/University_of_Calgary"
```

```
headers = {
    "User-Agent": "lab07-web-analyzer"
}

try:
    response = requests.get(url)
    response.raise_for_status()  # Ensures the request was successful
    soup = BeautifulSoup(response.text, 'html.parser')
    print(f"Successfully fetched content from {url}")
except Exception as e:
    print(f"Error fetching content: {e}")
```

The soup variable contains a BeautifulSoup object that represents the parsed HTML content of the fetched web page. It allows you to easily extract and manipulate HTML elements like headings, links, and paragraphs. Print the content of soup variable using the prettify() method:

```
print(soup.prettify())
```

3. Data Analysis

In web_analyzer.py, write Python code that counts and displays:

- The number of headings (<h1> to <h6>) in the HTML content. This should be a total count of all heading tags, rather than a count for each individual heading level.
- The number of links (<a> tags).
- The number of paragraphs (<p> tags).

Hint: You can use Beautiful Soup's find_all method.

4. Word Frequency Analysis

Analyze the webpage's content to identify the most frequently used words. To achieve this:

- Extract all the text content from the web page using soup.get_text().
- Convert the text to lowercase.
- Split the text into individual words using the findall method of Python's re (regular expressions) module. For example:

```
import re

words = re.findall(r'\b\w+\b', sometext)  # r'\b\w+\b' is a regular expression that matches sequen
ces of word characters (letters, digits, and underscores) that are surrounded by word boundaries
 (e.g., spaces or punctuation).
```

- Count the frequency of each word.
- Display the top 5 most frequently occurring words with their counts.

5. Keyword Search

Write Python code that asks the user for a keyword and counts how many times it appears in the webpage content, ignoring case. Then print the result clearly.

Hint: Use soup.get_text() to extract plain text from the parsed HTML, then convert both the extracted text and the user-entered keyword to lowercase before searching.

6. Finding the Longest Paragraph

Write Python code to find and display the longest paragraph in the webpage, and the number of words it contains. Ignore empty paragraphs or paragraphs with fewer than 5 words.

7. Visualizing Results

The next step is to visualize the extracted data using matplotlib library. First, we need to install matplotlib by running the following command:

```
pip install matplotlib
```

Next, create a bar chart showing the counts of headings, links, and paragraphs computed in step 3. Set the title of the chart to your group number and label the axes appropriately.

```
import matplotlib.pyplot as plt

labels = ['Headings', 'Links', 'Paragraphs']
values = [headings_count, links_count, paragraphs_count]

plt.bar(labels, values)
plt.title('Put your Group# Here')
plt.ylabel('Count')
#plt.savefig('web_analysis_results.png')  # Save the figure as an image file
plt.show()
```

Submission

- Fill out the names and UCIDs of all group members in Answer_sheet.
- Create a GitHub repository and push the code you wrote for this lab there.
- Upload the code you created for this lab to D2L in a single compressed zip file.